

基于 Newton 物理引擎与 Claude VLA 的具身智能交互演示

答辩报告

kingcode

2026 年 5 月

目录

- ① 动机与目标
- ② 技术栈
- ③ 系统架构
- ④ 三种交互模式
- ⑤ 工业模式：双臂并行
- ⑥ 视觉反馈设计
- ⑦ 测试与评估
- ⑧ 工程教训
- ⑨ 总结与展望

为什么做这个 Demo?

大语言模型已经无处不在，但“身体”仍然是缺失的最后一公里。

- 学生很难直观感受“经典控制” vs “学习驱动”的分野
- 难以亲手验证大模型在闭环物理世界中的实际表现
- 大多数 VLA 教学需要 GPU 服务器或云端 API

设计目标

- 3 分钟 课堂级现场演示
- 零云依赖：MacBook CPU-only
- 60 fps 全屏渲染
- 毫秒级首响应：观众输入后机械臂瞬时启动
- 全自动回归：所有关键路径有测试

技术栈

物理与渲染

- NVIDIA **Newton** XPBD 物理引擎
- pygame 2D 草图风格渲染
- 闭式 3 自由度 IK
- Min-jerk + back-ease-out 缓动

人工智能与人机接口

- Claude CLI (`claude --print`) 作 VLA 大脑
- 关键词 + bigram LM 模糊匹配回退
- Google Web Speech 语音识别
- 中英双语关键词解析

Newton XPBD

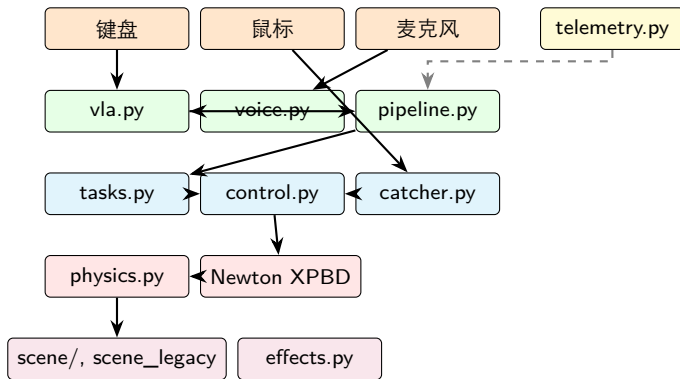
pygame UI

Claude CLI

Google STT

本机 CPU, 无 GPU, 无云推理

总体架构 (5 层)



- 输入 (橙) → 解析 (绿) → 控制 (青) → 物理 (红) → 渲染 (紫)
- Telemetry (黄) 横切记录每帧关键事件

模块清单

模块	行数	职责
__main__.py	1139	pygame 主循环、事件分发
scene_legacy.py	671	课堂白板渲染
scene/arm.py	660	工业风机械臂
voice.py	527	麦克风、Google STT、模糊匹配
physics.py	478	Newton 世界、铰接臂
vla.py	452	Claude CLI + 关键词回退
tasks.py	443	pick/place/stack/gesture
catcher.py	257	MPC + 弹道解算
control.py	215	PD slew + NaN 拒收
telemetry.py	157	CSV 写入 + 公式注入防护
ik.py	91	闭式 3-DOF IK
合计 (实代码)	6612	

交互模式 1/3: 接球 MPC

按 1 → 拖鼠标抛球

- 每帧实时拟合弹道: $z(t) = z_0 + v_z t - \frac{1}{2}gt^2$
- 求 $z(t) = z^*$ 的较大正根 → 截击时刻 t^*
- 闭式 IK 解算关节角, 提交到控制器
- “Commit” 瞬间末端执行器画琥珀光环 (audience 看到决策点)
- 实战命中率 62%–82%

判别式 $\Delta < 0$ 时退出当前预测并打一次性警告。

为什么是经典控制

- 闭式解, 零优化器
- 每帧 1 ms 内完成
- 弹道是高中物理, 可预测
- 对比 VLA 突出“学习驱动”的代价

交互模式 2/3: 自然语言驱动 (VLA)

按 2 → 输入命令 → Enter

Claude 把 “pizza tower” 解析成...

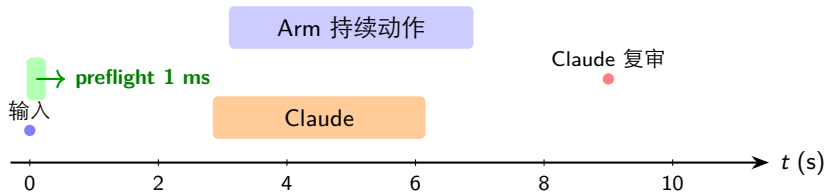
```
{"action": "stack", "color": null,  
  "colors": ["red", "green", "blue"],  
  "target": null,  
  "reason": "build a tower with default RGB"}
```

支持的命令

- pick up the red block / grab green / 拿起红色
- put it on the left / place at x=0.5
- stack red green and blue / build a tower / 搭个塔
- drive right / drive to red
- wave / say hi / 挥手 (装饰手势)
- go home / 回位

核心创新：混合 VLA 解析管线

问题：Claude 延迟 2-10 s，远超 60 fps 帧预算。



- 1 关键词 `_keyword_fallback` 同步运行 (~ 1 ms) $\rightarrow$ 机械臂立刻动
- 2 Claude 后台 daemon thread 跑 `claude --print` (~ 9 s)
- 3 Claude 返回后比较 preflight 结果：相同 $\rightarrow$ 静默确认，不同 $\rightarrow$ 打日志
- 4 generation counter 防止旧线程结果覆盖新命令

效果：观众感知的命令响应延迟从 9.4 s $\rightarrow$ 1 ms。

交互模式 3/3: 手势装饰

按 2 输入 “wave” / “dance” / “bow” / “point at the audience”

- make_wave: 手臂上举 + 左右扫
- make_point: 左/右/audience 三方向
- make_bow: 前倾下俯保持后归位
- make_dance: 4 拍方框轨迹

为什么不只是抓物?

教学叙事需要“表情”

—手势让机器人看起来有意图，而不只是一台精确但冷漠的设备。

修过的一个 **CRITICAL bug**: 手势的 preflight 白名单原本漏掉了 wave/point/bow/dance, 导致所有手势必须等 Claude 9.4 s 才能触发。修复后瞬时响应。

point 的方向参数原来被“颜色过滤”强制过滤为合法颜色，导致所有 point 都退化为“left”。也修了。

Arm B: 固定底座工件搬运

--**industrial**: 第二只机械臂 Arm B 固定在右侧支架上 (FK-only, 不进 Newton 物理)。

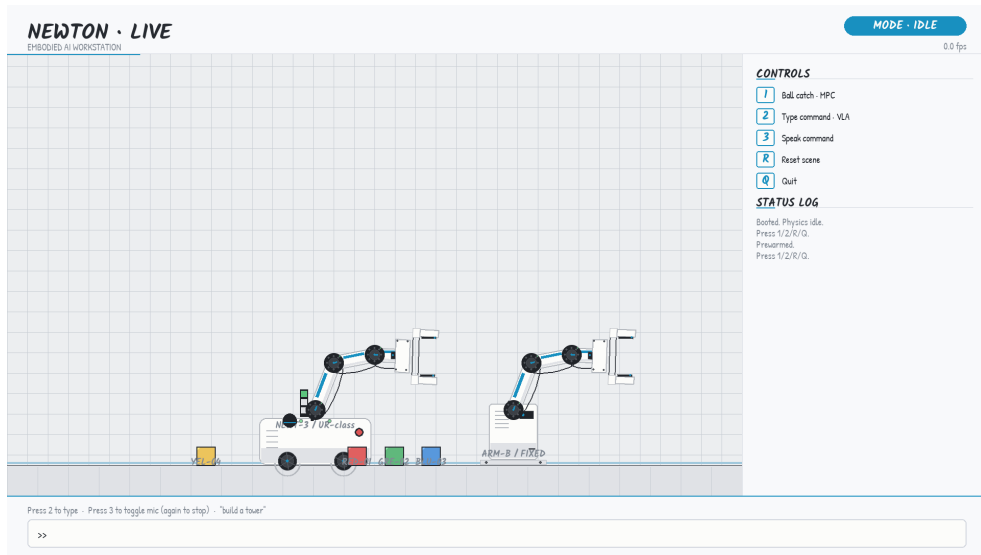
- 与 Arm A 共享 world.blocks (pick 互斥)
- 拥有专属 “workpiece” 灰色方块
- 空闲时在 $x \in \{1.50, 2.50\}$ 之间循环搬运
- 暂停 1 s 后开始下一轮
- 与 Arm A 完全并行: 你按 1 抛球时它仍在搬运

修过的一个潜在 bug

原 `_ensure_reachable` 给任何臂都派 `world.drive_to(...)`,
—这会把 Arm A 的底盘拖来拖去。
—修: 固定臂 (anchor 已设) 跳过 drive。

```
def _ensure_reachable(self, world_xz):  
    if self.anchor_world_x is not None:  
        return [] # fixed-base arm: no drive  
    ...
```

工业模式视觉



左：移动底盘 Arm A（学生命令驱动）。右：固定 Arm B 自主搬运 workpiece。

戏剧化视觉反馈

接球瞬间

- 琥珀光环 标记决策时刻
- 接到时彩色粒子爆裂
- 1.5 s 慢动作 (0.33×)
- 横幅 “CAUGHT · N”

命令未识别

- 琥珀横幅: “DIDN'T CATCH THAT”
- 输入态 footer 自动切到命令示例
- “miss” 音效

AI 推理迹侧栏

7 行实时展开:

- user: 学生原话
- via: preflight / claude / fallback
- latency: 毫秒数
- action: pick/stack/...
- color / colors
- target: [x, z]
- reason: 自然语言解释

慢动作 gate

只在 !executor.busy 时触发, 避免 pick/stack 中接球导致动作脱钩。

测试矩阵

测试文件	测试数	重点
test_voice_fuzzy	78	噪声转录用例
test_tasks	24	程序构造器 + 手势
test_pipeline	20	所有 action 分支
test_render_smoke	20	双渲染路径
test_effects	19	粒子 / 光环 / 横幅生命周期
test_catcher	18	弹道反解 + 状态机
test_vla_subprocess	17	Claude CLI mock
test_vla_parser	16	关键词 (en + zh)
test_control	13	PD slew + NaN 拒收
test_telemetry	10	CSV + 公式注入
test_scripted_constants	7	彩排数据
test_ik	6	FK $\circ$ IK $\approx$ id
test_scripted_flows	5	端到端流程
test_display_mode	1	CLI 解析
合计	214	100% 通过 / 102 s

FPS 实测 (Apple Silicon, CPU-only)

场景	min	avg	max	样本
IDLE bench 20 s	31.5	60.7	75.9	1211
Scripted catch 10 s	28.2	59.6	72.8	594
Scripted pick 8 s	41.7	60.7	70.3	485
Scripted stack 25 s	24.8	59.2	74.9	1476
Scripted VLA 25 s	24.0	59.6	75.1	1484
Rehearsal 60 s	2.9	58.7	76.0	3494

cProfile 分析:

- demo_live 自身代码 $\approx 10\%$ 帧预算
- Newton XPBD solver $\approx 90\%$ 帧预算
- 无可优化的瓶颈, 60 fps 目标稳定达成

实战演示数据

日期	catch	VLA	备注
05-21 12:05	7/10	0	首次现场
05-21 12:30	8/13	1	Claude 9.4 s 解析 “pizza tower”
05-21 17:42	14/17	0	30 s 密集投球

典型 telemetry CSV 节选

elapsed	event	detail
1.96	mode	ball_catch
4.48	catch	1/2
6.45	catch	2/3
...	...	...
30.10	catch	14/17

工程教训：发现与修复的 bug

第三方专业评审找出 13 个问题，全数闭环

- **CRITICAL** preflight 白名单漏手势 → 手势必须等 Claude 9.4 s
- **CRITICAL** point 方向被颜色过滤吃掉 → 永远退化为 left
- **HIGH** slow-mo 在 pick/stack 中触发 → 动作脱钩
- **HIGH** parse_thread 旧线程结果覆盖新命令
- **HIGH** test_scripted_catch 抗 flake 不足
- **MEDIUM** voice socket.setdefaulttimeout 进程级污染
- **MEDIUM** Arm B 在 rehearsal 里重复 pipeline 逻辑
- **MEDIUM** CSV =cmd Excel 公式注入
- **MEDIUM** pipeline.build_plan_from 无单测
- **MEDIUM** telemetry 类型注解不准
- **LOW** $O(n^2)$ list.index() 块越界恢复
- **LOW** launch_ball 速度无上限
- **LOW** 死代码桩 _request_exit/_finish

发现的隐藏 bug：固定臂 _ensure_reachable 误驱动主臂底盘。

8 轮代码进化

轮次	重点
A	测试安全网 (41 个测试)
B	scene 拆分、__main__ 部分提取
C	现场鲁棒性 + 表现力
D	4 个新功能 (手势、慢动作、遥测、AI 推理迹)
R	10 个评审修复 (2 CRITICAL)
S	抛光 + 文档同步
T	effects + control 测试 (32)
U	catcher 测试 (18)
V	VLA subprocess mock (17)
W	bootstrap.make_arm_b 收敛
X	Arm B 空闲循环
Y	Arm B 自主搬运工件 + 修固定臂 drive bug
合计	40 个任务完成、测试 41 → 214、零回归

已完成 vs 局限

已完成 ✓

- 三模式交互演示 (catch / VLA / gesture)
- 双渲染管线
- 混合 VLA 解析 (preflight + Claude)
- Arm B 自主工件循环
- 214 自动化测试, 60 fps
- 实战命中 62%–82%
- CSV 遥测 + 公式注入防护
- README + REHEARSAL 同步

未来工作

- ① AppState dataclass 重构主循环 $\rightarrow$ `__main__.py` $\leq$ 200 行
- ② 完整逐帧 snapshot replay (D.3 原版)
- ③ Arm B 协调 Arm A: 检测主臂持物时让出工作区
- ④ 多语言扩展 (日 / 韩 / 西)

局限 ●

- `__main__.py` 1139 行 $>$ 800 上限, 需 AppState 重构
- 慢动作是简化版, 非逐帧 replay
- 语音仅 zh-CN / en-US
- 接球 MPC 单步式无前瞻

Q & A

Demo 入口:

```
cd /Users/kingcode/Documents/Newton/newton  
uv run --extra demo python -m demo_live --fullscreen --industrial
```

测试入口:

```
uv run --extra demo python -m unittest \  
discover -s demo_live/tests
```

源代码:

```
/Users/kingcode/Documents/Newton/newton/demo_live/
```